parse each row into a hash with the column names as the keys and the row values as the values.

The library then reads the rest of the file, passing each row in turn to the block (the code between do and end). Inside the block, we extract the data from the ISBN and Price columns and use that data to create a new BookInStock object. We then append that object to an instance variable called @books_in_stock (the << operator does different things in Ruby, in this case, it means "append to an array"). And just where does that variable come from? It's an array that we created in the initialize method.

Again, this is the pattern you want to aim for. Your initialize method sets up an environment for your object, leaving it in a usable state. Other methods then use that state.

If you encounter an error along the lines of "'Float': can't convert nil into Float (TypeError)" when you run this code, you likely have extra spaces at the end of the header line in your CSV data file. The CSV library is pretty strict about the formats it accepts.

So, let's turn this from a code fragment into a working program. We're going to organize our source into three files. The first, book_in_stock.rb, will contain the definition of the class BookInStock. The second, csv_reader.rb, is the source for the CsvReader class. Finally, a third file, stock_stats.rb, is the main driver program. We'll start with book_in_stock.rb:

tut_classes/stock_stats/book_in_stock.rb

```
class BookInStock
  attr_reader :isbn, :price

  def initialize(isbn, price)
    @isbn = isbn
    @price = Float(price)
  end

  def price_in_cents
    (price * 100).round
  end
end
```

We're keeping the price_in_cents method so we can do money arithmetic without accumulating floating point errors.

Here's the csv_reader.rb file. The CsvReader class has two external dependencies: it needs the standard CSV library, and it needs the BookInStock class that's in the file book_in_stock.rb. Ruby has a couple of helper methods that let us load external files.

tut_classes/stock_stats/csv_reader.rb

```
require "csv"
require_relative "book_in_stock"

class CsvReader
  def initialize
    @books_in_stock = []
  end

  def read_in_csv_data(csv_file_name)
    CSV.foreach(csv_file_name, headers: true) do |row|
      @books_in_stock << BookInStock.new(row["ISBN"], row["Price"])
    end
  end
```